

Autorzy Projektu

Mateusz Kowalski (Sekcja: 2)

Paweł Kowolik (Sekcja: 2)

Damian Feter (Sekcja: 2)

1. Opis Zrealizowanego Projektu

Terminal Arcade to wielozadaniowa, w pełni zorientowana obiektowo aplikacja konsolowa napisana C++, stanowiąca zbiór dziewięciu klasycznych i zręcznościowych gier. Projekt łączy elastyczność programowania obiektowego z wydajnym niskopoziomowym renderowaniem w konsoli tekstowej.

Główne cele projektu:

- Unifikacja sterowania:** Wszystkie gry oraz menu główne są obsługiwane spójnie za pomocą klawiszy `W/A/S/D` (lub strzałek kierunkowych), wyboru dokonuje się poprzez `SPACJĘ` lub `ENTER`, a powrót/wyjście realizowane jest klawiszem `Q`.
 - Czystość kodu i zgodność ze standardami:** Całość kodu źródłowego została udokumentowana zgodnie ze standardem Doxygen, co ułatwia automatyczne generowanie dokumentacji technicznej i poprawia czytelność.
 - Optymalizacja wydajności (Flicker-Free):** Zaimplementowano autorski system buforowania ramki zapobiegający migotaniu ekranu (częstego problemu konsolowy wykorzystujący `system("cls")` do zmiany treści na ekranie).
-

2. Architektura i Założenia Obiektowe

Aplikacja została zaprojektowana zgodnie z paradygmatami programowania zorientowanego obiektowo (OOP):

- Dziedziczenie i Polimorfizm:**

Stworzono abstrakcyjną klasę bazową `Game`, która definiuje interfejs dla każdej gry (wirtualna metoda `play()`, pobieranie nazwy, opisu oraz konfiguracja zapisu wyników). Każda z dziewięciu gier (np. `TicTacToe`, `MineSweeper`, `Solitaire`) dziedziczy po klasie `Game` i dostarcza własną, specyficzną implementację logiki rozgrywki oraz rysowania planszy.

Menu główne `Arcade` zarządza kolekcją gier poprzez wektor inteligentnych wskaźników typu `std::vector<std::shared_ptr<Game>>`, realizując wywołania polimorficzne w czasie wykonywania programu.

1. Wzorce Projektowe:

Singleton: Klasa `Database` reprezentująca plikową bazę danych wyników (`scores.txt`) została zaimplementowana przy użyciu wzorca Singleton. Gwarantuje to jedyny, globalny punkt dostępu do operacji zapisu i odczytu wyników z dowolnego miejsca w aplikacji.

1. Separacja Logiki i Prezentacji:

Wydzielono klasę `Display`, która odpowiada wyłącznie za renderowanie kolorowych znaków ANSI oraz stabilizację obrazu.

Wydzielono moduł `Input`, który w sposób przenośny (zależny od systemu operacyjnego Windows/POSIX) odczytuje klawisze w trybie nieblokującym.

3. Instrukcja Użytkownika i Sterowanie

Ogólne zasady nawigacji:

Ruch / Poruszanie: Klawisze `W`, `A`, `S`, `D` lub `Strzałki`

Wybór / Akcja: `Spacja` lub `Enter`

Powrót / Wyjście: Klawisz `Q`

Sterowanie i zasady w poszczególnych grach:

1. Tic-Tac-Toe (Kółko i Krzyżyk)

Cel: Ułóż 3 swoje symbole w pionie, poziomie lub po przekątnej.

Sterowanie: Poruszaj się kursorem po powiększonej planszy za pomocą WASD, zatwierdzaj postawienie znaku (`X` lub `O`) klawiszem `SPACE` / `ENTER`.

Tryby: Gracz vs Gracz lub Gracz vs BOT (3 poziomy trudności: Łatwy, Średni, Trudny - algorytm Minimax).

2. MineSweeper (Saper)

Cel: Odkryj wszystkie pola, na których nie ma min.

Sterowanie: Ruch kursorem za pomocą WASD. **SPACE** odkrywa pole (lub wykonuje szybkie odkrycie sąsiadów "chording" jeśli liczba oznaczonych flag zgadza się z liczbą na polu). Klawisz **F** przełącza oznaczenie (Ukryte -> Flaga **F** -> Pytajnik **?**).

3. 2048

Cel: Łącz sąsiednie kafelki o tej samej wartości, aby uzyskać kafelek o wartości 2048.

Sterowanie: Przesuwaj całą planszę za pomocą **W/A/S/D** / **Strzałek** .

4. Blackjack (Oczko)

Cel: Zbierz karty o łącznej wartości jak najbliższej 21, nie przekraczając jej i pokonując krupiera.

Sterowanie: Wybierz stawkę, a następnie decyduj: **H** (Hit - dobierz kartę) lub **S** (Stand - pasuj).

5. Battleship (Statki)

Cel: Zatop wszystkie ukryte statki przeciwnika.

Sterowanie: Rozmieść statki ręcznie za pomocą **W/A/S/D** i **R** (obróć) lub wybierz automatyczne rozmieszczenie. Podczas bitwy celuj kursorem **W/A/S/D** i oddawaj strzały klawiszem **SPACE** .

6. Snake

Cel: Zjadaj jedzenie (*****), rośnij i unikaj zderzeń ze ścianami oraz własnym ogonem.

Sterowanie: Zmieniaj kierunek ruchu węża w czasie rzeczywistym klawiszami **W/A/S/D** .

7. Solitaire (Pasjans)

Cel: Ułóż cztery kompletne stosy kart od Asa do Króla w polach fundamentów.

Sterowanie: Poruszaj się kursorem po planszy za pomocą **W/A/S/D** . Naciśnięcie **SPACE** zaznacza kartę/stos, a ponowne naciśnięcie **SPACE** na innej kolumnie wykonuje ruch zgodnie z zasadami Klondike.

8. Bomberman (PvP)

Cel: Pokonaj drugiego gracza na generowanej arenie za pomocą eksplodujących bomb.

Sterowanie:

Gracz 1: **W/A/S/D** (Ruch) + **SPACE** (Bomba)

Gracz 2: **I/J/K/L** (Ruch) + **ENTER** (Bomba)

9. Cyber Defense (Tower Defense)

Cel: Broń serwerów przed nadchodzącymi wirusami.

Sterowanie: Poruszaj się kursorem **W/A/S/D** po liniach obrony, kupuj i stawiaj wieże (**1** - Górnik bitów, **2** -Działko, **3** -Firewall).

4. Ciekawostki Implementacyjne

1. **System Flicker-Free (Buforowanie Ekranu):** Większość konsolowych gier miga, ponieważ używają `system("cls")`, co zmusza system operacyjny do ponownego rysowania całego okna. W naszym projekcie klasa `Display` przekierowuje strumień `std::cout` do wewnętrznego obiektu `std::stringstream` (bufor klatki). Dopiero po zakończeniu rysowania ramki, bufor jest przetwarzany: do każdej linii dopisywana jest sekwencja ucieczki `\033[K` (czyszczenie linii od kursora do końca), po czym kursor powraca na pozycję początkową za pomocą `\033[H`. Zapewnia to optymalne ~20 FPS bez żadnego migotania.
 2. **Wykrywanie Kodu Konami (Easter Egg):** Menu główne posiada ukryty nasłuch sekwencji klawiszy. Wpisanie w menu kodu: `W W S S A D A D B A` (odpowiednik Konami Code: Góra, Góra, Dół, Dół, Lewo, Prawo, Lewo, Prawo, B, A) aktywuje ukryty **Tryb Tęczowy (Rainbow Mode)**. Powoduje on, że ramki i teksty menu głównego płynnie zmieniają kolory przy każdym ruchu.
 3. **Przenośność Wejścia (Cross-Platform Non-blocking Input):** Dla systemu Windows wejście klawiatury realizowane jest przez lekki nagłówek `<conio.h>` i funkcje `_kbhit()` oraz `_getch()`. Dla systemów Linux/POSIX zaimplementowano własną emulację tych funkcji w pliku `Input.h` za pomocą modyfikacji struktury `termios` i manipulacji flagami nieblokującymi `fcntl` na deskrytorze standardowego wejścia `STDIN_FILENO`.
-

5. Zgodność z Załoženiami i Wnioski

Podczas realizacji projektu ściśle przestrzegano wytycznych:

Walidacja danych: Każda metoda pobierająca dane wejściowe (np. stawki w Blackjacku, współrzędne przy budowaniu wież czy rozmiary niestandardowe w MineSweeperze) rygorystycznie sprawdza granice przedziałów i poprawność typów, zapobiegając awarii aplikacji.

Wykorzystanie OOP: Dziedziczenie klas gier po klasie bazowej `Game`, dynamiczny polimorfizm w menu oraz wzorce projektowe pokazują pełne wykorzystanie potencjału C++.

Komentarze i Dokumentacja: Kod został oczyszczony z przestarzałych komentarzy i w pełni udokumentowany strukturami Doxygen, co ułatwia generowanie profesjonalnej dokumentacji technicznej.

Testowanie: Dodano przełącznik `--test` do pliku wykonywalnego, umożliwiający natychmiastowe uruchomienie testów automatycznych logiki gier.

Możliwości rozbudowy:

1. **Tryb sieciowy (Multiplayer):** Wykorzystanie biblioteki gniazd sieciowych (np. ASIO) do rozbudowy gry Bomberman lub Statki do rozgrywki przez sieć lokalną.
2. **Dodanie efektów dźwiękowych:** Integracja prostej biblioteki audio do odtwarzania 8-bitowych efektów dźwiękowych w grach.
3. **Zapis stanu gry:** Rozbudowa bazy danych o możliwość zapisu stanu pasjansa lub trwającej kampanii w Cyber Defense.
4. **Dodanie kolejnych gier:** Rozbudowa o nowe klasyczne gry, np. Tetris, Arkanoid czy Pac-man.